Dipartimento di Elettronica e Informazione
Politecnico di Milano
prof. ***Carlo Ghezzi***
prof. ***Angelo Morzenti***
prof. ***Pierluigi San Pietro***
20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (39) 02-2399.3400
Fax (39) 02-2399.3411

Appello di Ingegneria del Software

13 Luglio 2004

Cognome

Nome

Matricola

Sezione (segnarne una) Ghezzi □, Morzenti □, SanPietro □

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h15m.
6. Punteggio totale a disposizione: 26/30.

Valutazione:
Esercizio 1: ……..

Esercizio 2: ……..

Esercizio 1 (punti 10)

1. Utilizzando opportune formule che definiscono le pre e post-condizioni, si fornisca la specifica di un'astrazione procedurale che in input riceve due array x e y di interi.
    a) L'astrazione restituisce true se x è una permutazione di y, false altrimenti, sotto l'ipotesi che ne' x ne' y abbiano elementi duplicati. Se almeno uno dei due array è null, allora viene lanciata l'eccezione unchecked NullPointerException.

Es. x = [1,5, 2], y = [2, 5, 1] risultato = true
x = [1,5, 2], y = [2, 5, 0] risultato = false
x = [1,5, 2], y = [2, 5, 2] risultato = indefinito

```
public static boolean risultato(int[]x, int[]y) {

//REQUIRES: forall i,j, 0<=i<j<x.size()-1, x[i]!= x[j] && forall i,j, 0<=i<j<y.size()-1, y[i]!= y[j]
//EFFECTS: if (x==null) || (y==null) throw NullPointereException
//          else if (x.size<>y.size) return false
//          else if x.size()==0 return true
//          else return
//                    (forall i, 0<=i<x.size()-1, exists j, 0<=i<x.size()-1, x[i] = y[j]) &&
//                    (forall i, 0<=i<y.size()-1, exists j, 0<=i<y.size()-1, y[i] = x[j])
```

2. In base alla specifica fornita, si definiscano cinque dati numerici significativi a supporto del test funzionale (black-box testing) in base ai criteri di copertura delle combinazioni proposizionali e dei valori limite, specificando inoltre i risultati attesi per ciascun dato.

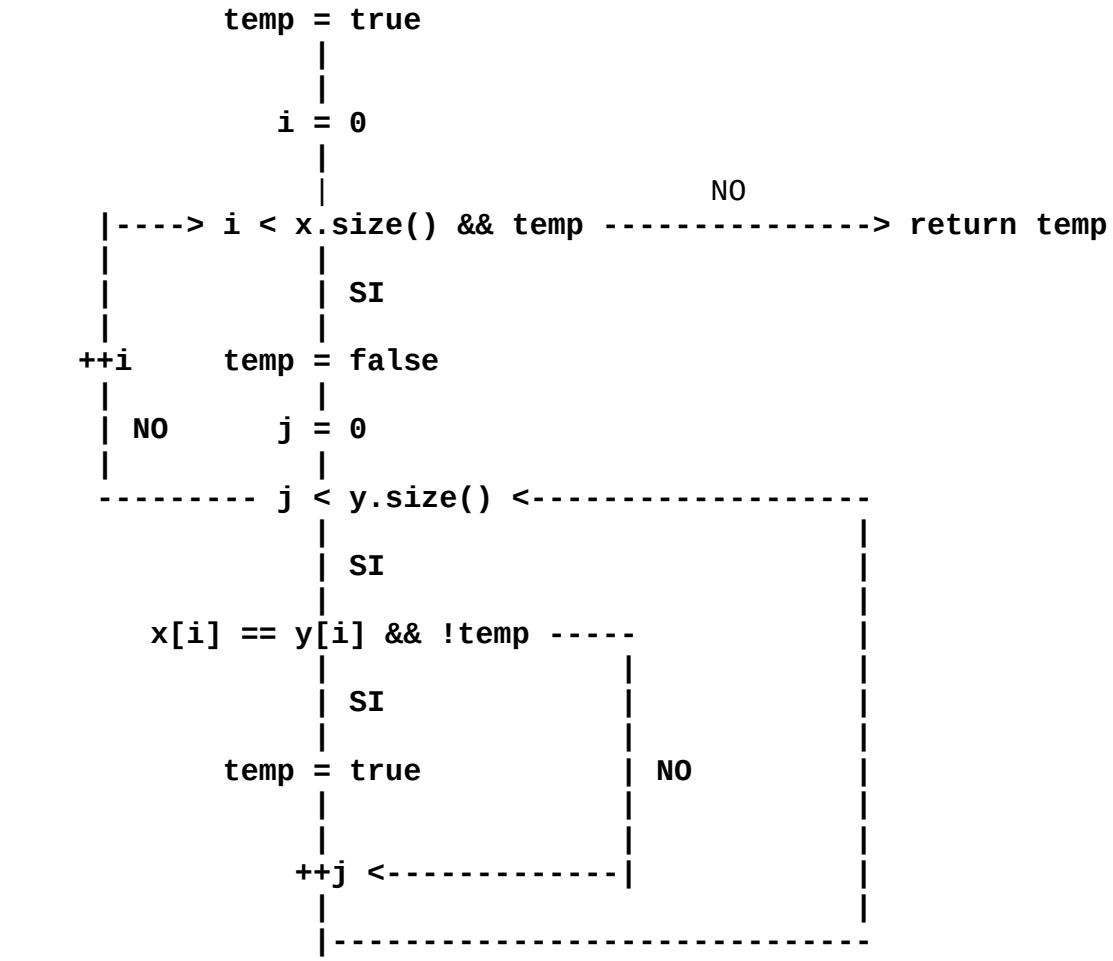
< x, y >	
<[null], [1, 2]>	**solleva NullPointerException**
<[1, 2, 3], [4, 5]>	**false**
<[], []>	**true**
<[3, 7, 1, 6], [1, 6, 7, 3]>	**true**
<[2], [2]>	**true**
<[1, 2, 3], [4, 5, 6]>	**false**

3. Si consideri la seguente implementazione del caso (a) (*non necessariamente corretta* rispetto alla specifica). Si identifichi il minimo numero di dati di test, specificandone valore numerico e risultato atteso, per garantire la copertura al 100% del codice secondo il criterio delle diramazioni.

```
public static boolean risultato(int[]x, int[]y) {
    boolean temp=true;
    for (int i=0; i < x.size() && temp; ++i) {
        temp= false;
```

```
        for (j=0; j<y.size(); ++j)
            if (x[i] ==y[j] && !temp) temp = true;

    }
    return temp;
}
```



Basta un solo caso di test, ad esempio <[5], [3, 5]>, per il quale il valore atteso secondo la specifica dovrebbe essere false, ma per il quale la funzione ritorna true.

Esercizio 2 (punti 16)

Si consideri la seguente specifica dell'astrazione sui dati PilaDoppiaTesta.

```
public class PilaDoppiaTesta {
/*OVERVIEW: Tipo mutabile, che rappresenta una pila con due estremità (sinistra e destra), che consente
l'inserimento sia in testa a sinistra che in coda a destra. L'estrazione avviene esclusivamente dalla testa di
sinistra, mantenendo l'ordine di inserzione. Gli elementi inseriti sono di tipo Object, ma diversi da null.
Un oggetto tipico è  [o1, o2, …, on], dove o1 e' la testa sinistra della pila, on è la testa destra.
*/

    public PilaDoppiaTesta ()
        //EFFECTS: costruisce la pila vuota
    public void pushLeft (Object x)
        //REQUIRES: x<>null
        //EFFECTS: posto this = [o1, o2, …, on], n>=0, this_post = [x, o1, o2, …, on]
        //MODIFIES this

    public void pushRight (Object x)
        //REQUIRES: x<>null
        //EFFECTS: posto this = [o1, o2, …, on], n>=0, this_post = [o1, o2, …, on, x]
        //MODIFIES this

    public Object pop() throws EmptyException;
        //EFFECTS: if this è vuota throw EmptyException
        //          else, posto this = [o1, o2, …, on], this_post = [o2, ..., on] e o1 è restituito
        //MODIFIES this

    public Iterator LeftToRight();
        //EFFECTS: restituisce un generatore che itera su tutti gli elementi di this,
        //          nell'ordine da sinistra a destra
        //REQUIRES: this non può essere modificato mentre il generatore è in uso.

    public Iterator RightToLeft();
        //EFFECTS: restituisce un generatore che itera su tutti gli elementi di this,
        //          nell'ordine da destra a sinistra
        //REQUIRES: this non può essere modificato mentre il generatore è in uso.

    public int size(); //EFFECTS: restituisce il numero di elementi nella pila
}
```

(**continua esercizio 2)**

a) Si specifichi, come sottoclasse di PilaDoppiaTesta, una variante in cui il metodo PushLeft lancia un'eccezione checked di tipo InvalidObject qualora sia passato un argomento x che vale null. La variante rispetta il principio di sostituzione?

```
public VariantePilaDoppiaTesta extends PilaDoppiaTesta {
    public void pushLeft (Object x) throws InvalidObject
        //EFFECTS: if (x == null) throw new InvalidObject else
        //posto this = [o1, o2, …, on], n>=0, this_post = [x, o1, o2, …, on]
        //MODIFIES this
```

Non rispetta il principio perche' viola la regola delle signature

(**continua esercizio 2)**

b) Si specifichi, come sottoclasse di PilaDoppiaTesta, un ADT DoppiaPila che possiede un'operazione supplementare popRight(), che è come pop ma estrae e restituisce l'elemento più a destra nella pila.
DoppiaPila rispetta il principio di sostituzione?

```
public class DoppiaPila extends PilaDoppiaTesta {
    //OVERVIEW: è una PilaDoppiaTesta in cui è possibile estrarre oggetti anche dalla testa di destra.
    public Object popRight() throws EmptyException;
    //EFFECTS: if this è vuota throw EmptyException
    //     else, posto this = [o1, o2, …, on], this_post = [01, ..., on-1] e on è restituito
    //MODIFIES this
```

NON rispetta il principio di sostituzione perché viola banalmente la proprietà dell'Overview di PilaDoppiaTesta: "L'estrazione avviene sempre dalla testa di sinistra". Quindi, sorprenderebbe tutti gli utilizzatori che potrebbero non recuperare con un pop un elemento inserito, pur ritrovando con pop() tutti gli elementi da loro inseriti prima di quello.

(**continua esercizio 2)**

c) Si completi la seguente implementazione della classe PilaDoppiaTesta, specificando AF e RI. Note: Il metodo iterator() dei Vector restituisce un generatore agli elementi a partire dalla posizione 0. Non vi è un simile metodo predefinito che restituisce un generatore dall'ultima posizione alla posizione 0: il metodo RightToLeft() va quindi implementato ad hoc, definendo anche la classe del generatore.
L'implementazione è corretta rispetto alla specifica? Giustificare la risposta.

```
public class PilaDoppiaTesta {
    Vector v; //rep
//AF(c) = [v(0), v(1), ... v(v.size()-1)].....
//RI(c) = v != null && forall i, 0<=i<v.size(), v(i) != null .....
    public PilaDoppiaTesta () {v = new Vector();}
    public void pushLeft (Object x) {
        v.add(x,0); //aggiunge x nella prima posizione di v
    }
    public void pushRight (Object x){
        v.add(x); //aggiuge x nell'ultima posizione di v
    }
    public Object pop() throws EmptyException {;
        if (v.size() ==0)  throw new EmptyException;
        Object o = v.elementAt(0);
        v.removeElementAt(0);
        return o;
    }
    public int size() {
        return v.size();
    }
    public Iterator LeftToRight() {
        return v.iterator();
    }
    public Iterator RightToLeft(){
    .....return new Iterator{
    .....  int pos = v.size()-1;
    .....  public boolean hasNext(){ return pos >= 0; }
    .....
    .....  public Object next() throws NoSuchElementException {
    .....    if (!hasNext()) throw new NoSuchElementException();
    .....    Object res = v.elementAt(pos);
    .....    pos--;
    .....    return res;
    .....   }
    .....
    .....  public void remove() throws UnsupportedOperationException {
    .....    throw new UnsupportedOperationException()
```

..... **}**

}

L'implementazione E' CORRETTA rispetto alla specifica, in quanto:

1- **l'implementzione conserva il RI, infatti il costruttore lo conserva, e i metodi, se invocati in maniera che la clausola REQUIRES sia soddisfatta, anche lo conservano**
2- **le operazioni sono corrette rispetto all'astrazione**
3- **se i metodi vengono invocati in modo da rispettare le clausole REQUIRES, le proprietà dell'astrazione (in particolar modo il fatto che gli oggetti nella coda siano diversi da null) vengono anche rispettate (le altre proprietà sono banalmente rispettate).**

(**continua esercizio 2)**

d) Si desidera ora utilizzare la classe PilaDoppiaTesta per implementare una PilaSenzaSimili di Object, che si comporta come una pila qualunque ma in cui non è possibile inserire oggetti "simili" a oggetti già presenti in Pila. Per stabilire se due oggetti sono simili, la Pila utilizza l'approccio related subtype, tramite il metodo simili di un'interfaccia Verificatore. Il Verificatore da utilizzare è stabilito alla chiamata del costruttore di PilaSenzaSimili. Le specifiche di PilaSenzaSimili e del Verificatore sono:

```
public class PilaSenzaSimili {
//OVERVIEW: una Pila di Object in cui non si possono inserire elementi "simili" a quelli presenti nella pila.
    public PilaSenzaSimili (Verificatore v)
    // REQUIRES: v != null
    // EFFECTS: costruisce una pila vuota
    public Object pop()//EFFECTS: restituisce ed elimina da this l'elemento in cima
    public void push(Object o) throws SimilarException
    //EFFECTS: if o è simile a uno qualunque degli elementi già presenti nella Pila, lancia SimilarException,
    //else inserisce o in cima alla pila; v
}
```

```
public interface Verificatore {
    public boolean simili(Object o1, Object o2);
    //EFFECTS: return true se o1 e o2 sono simili, false in tutti gli altri casi.
    //Devono valere le proprietà riflessiva e simmetrica:
    // se o1.equals(o2) allora deve valere simili(o1,o2);
    // se simili(o1,o2) allora simili(o2,o1).
}
```

Completare l'implementazione seguente della classe PilaSenzaSimili.

```
public class PilaSenzaSimili {
//AF(c) = AF(p).....
//RI(c) = p<>null && v!= null && forall Object x,y in p, (v.simili(x,y) ==> x==y).....
    private PilaDoppiaTesta p;
    private Verificatore v;
    public PilaSenzaSimili(Verificatore v) (p = new PilaDoppiaTesta(); this.v = v;}
    public Object pop() {return p.pop();}
    public void push(Object o) throws SimilarException {
      for (Iterator i = p.leftToRight(); p.hasNext();) {
          Object o1 = p.next();
          if (v.simili(o,o1)) throw new SimilarException();
      }
        .....
        .....
        .....
```

```
        .....
        p.pushLeft(o);
    }
}
```

Considerare poi la classe Double, corrispondente al tipo primitivo double. Si codifichi una classe RifiutaDoubleVicini che implementi l'interfaccia Verificatore e permetta di inserire oggetti nella Pila con il seguente criterio di similitudine: due oggetti x1 e x2 sono simili se entrambi x1 e x2 sono Double e inoltre vale Math.abs (x1-x2)<0.00001, oppure (nel caso in cui non siano entrambi Double) se x1 e x2 sono "equals". Mostrare un esempio di costruzione di una pila.

```
public class RifiutaDoubleVicini implements Verificatore {
    public boolean simili(Object o1, Object o2) {
    //EFFECTS: return true se o1 e o2 sono simili, false in tutti gli altri casi
    if (!(o1 instanceof Double && o2 instanceof Double) return o1.equals(o2);
    double d1 = ((Double) o1).value();
    double d2 = ((Double) o2).value();
    if (Math.abs(d1-d2)<0.00001)
        return true;
    else return false;
}
....
PilaSenzaSimili p = new PilaSenzaSimili (new RifiutaDoubleVicini ());
....
```